

SiteFlow, start to end

A step-by-step study of the construction project-controls assistant. Each step tells you what to do, then explains where that work sits in the system, why it is built this way, and the tradeoff. The reference code is in `projects/siteflow`. Study one step, run its check, then go on. Ollama may be installed. It is not used until Step 24. The AWS account may exist. It is not used until you choose Step 26.

The one sentence for the whole project:

```
The React app never calls the model or the file bucket.  
It calls FastAPI. FastAPI runs a LangGraph supervisor  
that picks one specialist, and pauses if the answer  
would change a buy or the schedule.
```

Contents

- Step 1: Say what you are building before you write a file
- Step 2: Draw the picture, then keep every file inside it
- Step 3: Create the folder and a Python environment
- Step 4: Confirm Ollama, and do not call it yet
- Step 5: Create the AWS account, then stop
- Step 6: Write `calculate_eoq` in `backend/app/tools/inventory.py`
- Step 7: Write `safety_stock` in the same file
- Step 8: Write `lookup_sku` and `load_materials_csv`
- Step 9: Prove the math with `tests/test_inventory.py`
- Step 10: Cut specifications into chunks in `backend/app/rag/chunking.py`
- Step 11: Rank chunks with BM25 in `backend/app/rag/retriever.py`
- Step 12: Turn a PDF into chunks in `backend/app/rag/ingest.py`
- Step 13: Store the original file in `backend/app/storage/local.py`
- Step 14: Open the first HTTP door: upload and list
- Step 15: Finish Day 1 by running the file tests together
- Step 16: Define the shared turn in `backend/app/agents/state.py`
- Step 17: Write the supervisor cascade in `backend/app/agents/supervisor.py`
- Step 18: Write the spec agent so it can only quote the project
- Step 19: Write the materials agent so it will not guess a missing input
- Step 20: Write the risk agent as rules over explicit lines
- Step 21: Pause for a person with LangGraph interrupt
- Step 22: Expose chat and approvals over HTTP
- Step 23: Score the 12 gold questions in `eval/questions.json`
- Step 24: Point the sentence writer at your local Ollama
- Step 25: Build the React screen that only calls FastAPI
- Step 26: Add the S3 and IAM seam without pretending you deployed it
- Step 27: Run the three demo questions and say the walk-through out loud

Day 1 is Steps 6 through 15: math, chunks, retrieval, storage, and upload. Day 2 is Steps 16 through 22: state, supervisor, three specialists, the pause, and the chat API. Day 3 is Steps 23 through 27: eval, Ollama, the screen, the AWS seam, and the spoken walk-through. Steps 1 through 5 are the setup you do before any of that code.

Step 1: Say what you are building before you write a file

Do this.

Write one sentence on paper: SiteFlow helps one construction job. A person uploads specs, an RFI, and a materials CSV. They ask a question. A supervisor sends it to document search, inventory math, or schedule risk. If the answer would change a buy or the schedule, the screen waits for Approve or Reject.

Where this sits.

This sentence is the whole product. Every later file is a piece of it. If a file does not serve this sentence, it does not belong in the three-day slice.

In depth.

A project engineer cares about three kinds of questions. 'What does the steel spec say about lead time?' is a document question. 'What EOQ should we use for rebar?' is a math question. 'Which open RFI blocks the foundation pour?' is a schedule question. SiteFlow is those three paths plus a pause before anything expensive. It is a practice copy of the shape Autodesk Construction Cloud interviews ask about: documents, project data, and a human gate. It does not log into Autodesk, and it does not create a real RFI.

Tradeoff.

A smaller demo would be one chatbot over PDFs. That is easier, and it hides the thing the interview wants: routing, tools, and a human gate. A larger demo would be five microservices and Kubernetes. That looks like production and teaches none of the agent behavior. The thin slice is the better study object because you can point at every hop.

Check.

You can say the sentence without looking at the screen. You have not opened LangGraph, React, or the AWS console yet.

Step 2: Draw the picture, then keep every file inside it

Do this.

Draw this stack and leave it where you can see it. Browser on port 5173. FastAPI on port 8000. Under FastAPI: files on disk, a library of chunks, a LangGraph supervisor, and a SQLite checkpoint. The browser has one rule: it only sends JSON to FastAPI.

Where this sits.

The picture is the map for the rest of the PDF. Day 1 builds the math and the files. Day 2 builds the graph. Day 3 builds the screen and the AWS versions of the same jobs.

In depth.

The browser is the product. FastAPI is the platform. The product asks for an upload, a chat answer, or an approval. The platform decides how that happens. A later screen can change without touching the agents. The agents can change without the screen learning S3 or LangGraph. One process is also the place that rejects a bad file and the place that keeps a Python traceback off the screen. S3, Bedrock, and DynamoDB sit beside this picture. They store a file, write a sentence, and save a checkpoint. They are the same jobs with AWS authentication. They are not on the path you run today.

Tradeoff.

Letting the browser call S3 and the model directly removes a hop, which feels faster. Then every secret and every 'may this user see project B?' check lives in JavaScript. You cannot defend that split in a platform interview. The extra hop is the design.

Check.

Point at the picture and name the only arrow that leaves the browser. It ends at FastAPI.

Step 3: Create the folder and a Python environment

Do this.

From the repo root, enter `projects/siteflow/backend`. Create a virtual environment, activate it, and install `requirements-dev.txt`. That file pins FastAPI, LangGraph, pypdf, pytest, and the SQLite checkpointer to the versions the tests ran against.

Where this sits.

The environment is the workbench. Nothing in it is the product yet. Pinning versions means a later install does not silently change LangGraph's interrupt behavior.

In depth.

Run: `cd projects/siteflow/backend && python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements-dev.txt`. The app code lives in `backend/app`. Tests live in `backend/tests`. Sample PDFs live in `data/sample`. The study notes live in `docs`. Runtime uploads and the SQLite file go in `data/runtime`, which git ignores, so a local demo never gets committed. `requirements.txt` is what a server needs. `requirements-dev.txt` adds `pytest`.

Tradeoff.

Installing the latest of every package on each machine is convenient and makes tutorials drift. A pin can go stale. For a study repo that you will defend line by line, the pin is the right call: you know which LangGraph you explained.

Check.

`python -c "import fastapi, langgraph, pypdf; print('ok')"` prints ok inside the virtualenv.

Step 4: Confirm Ollama, and do not call it yet

Do this.

On your machine run `ollama list`. If `llama3.1:8b` is missing, run `ollama pull llama3.1:8b`. Then close that terminal. Day 1 does not send a prompt.

Where this sits.

Ollama is the local sentence writer you will plug in at Step 24. The formulas and the file checks have to be true before a model is allowed to phrase them.

In depth.

`llama3.1:8b` is the model named in the project guide. It runs on your laptop, so a study session does not spend Bedrock money. SiteFlow talks to it through one method, `invoke(messages)`, in `backend/app/agents/llm.py`. The same method exists for a template writer and for Bedrock. Until the math tests are green, the default writer is the template: it can only repeat the draft the tools already computed. A model that writes the EOQ sentence first will sound sure and can still be wrong.

Tradeoff.

Calling the model on day 1 feels like progress because you see prose. You cannot tell a bad formula from a bad prompt. Waiting is slower to demo and much easier to debug. Use the model to polish a draft that already has the number 547.72, not to invent the number.

Check.

ollama list shows llama3.1:8b. You have not imported OllamaChatModel from Day 1 code.

Step 5: Create the AWS account, then stop

Do this.

Open <https://aws.amazon.com/free/> and create a free account. Turn on root multi-factor authentication. Choose the region US West (Oregon), us-west-2. Do not create a bucket and do not download access keys.

Where this sits.

AWS is Day 3. The account is only a place you will later put files (S3), an optional model (Bedrock), and an optional checkpoint (DynamoDB). Day 1 and Day 2 run entirely on the laptop.

In depth.

Use an email you control, a unique password, and an account name like siteflow-YOURNAME. Enter a real address and phone number so AWS can send the verification code. Free tier still requires a card. After you can sign in, create a billing alarm for a few dollars so a surprise charge emails you. Choose Basic support, the free plan. Sign in once as root only to turn on MFA. After that, daily work is a separate IAM user. Do not attach AdministratorAccess. The policy you will attach later is `infra/iam-siteflow.json`, and only after the bucket exists. Keys, when you eventually make them, go in `backend/.env`. That file is `gitignored`. `backend/env.example` shows the names and contains no secrets.

Tradeoff.

Skipping the account keeps the laptop demo pure, and then the AWS story in an interview is theoretical. Creating the account now and using it now mixes billing, regions, and IAM into the formula bugs you are trying to see clearly. Create it, then leave it idle.

Check.

You can sign in to the console in us-west-2. You have no access key on disk.

Step 6: Write `calculate_eoq` in `backend/app/tools/inventory.py`

Do this.

Create a plain function `calculate_eoq(annual_demand, order_cost, holding_cost)`. It returns $EOQ = \sqrt{2 * D * S / H}$, rounded to two decimals, plus orders per year, the three inputs, and the formula string. If any input is less than or equal to zero, return a dict with an error key and no eoq key.

Where this sits.

This function is the materials specialist's calculator. The specialist does not exist yet. Nothing here calls Ollama, FastAPI, or S3. Every layer above will repeat this number, so it has to be checkable by hand.

In depth.

D is how many units the job needs in a year. S is the cost to place one order. H is the cost to hold one unit for a year. Order too often and you pay S over and over. Order a huge pile and you pay H all year. The square root is the order size that balances those two costs when demand is steady. For the rebar question, D is 12000 and H is 4. The question forgets S. This function does not guess S. A later step may fill S from the one matching CSV row, REBAR-5, whose order cost is 50. Then $\sqrt{2 * 12000 * 50 / 4} = \sqrt{300000} = 547.72$. `orders_per_year` is D / EOQ , about 21.91. That is how many orders, not a dollar total. Unit cost in the CSV is 4.10. That is what one bar costs to buy. Holding cost is what it costs to keep one bar for a year. The function has no path that copies 4.10 into H. Returning a dict instead of raising lets a later chat show the error sentence. Echoing D, S, H, and the formula lets a superintendent audit the answer without reading Python.

Tradeoff.

A fancier inventory model handles quantity discounts, one-time buys, and a lead time that jumps every week. You cannot recompute that on a whiteboard, and Project 2 does not use it. Classic EOQ is the right teaching formula and the wrong tool for a single pour. `round(..., 2)` is for a person. Two close inputs can display as the same quantity. Say both limits out loud.

Check.

`calculate_eoq(12000, 50, 4)['eoq']` is 547.72. `calculate_eoq(12000, 50, 0)` has an error and no eoq.

Step 7: Write `safety_stock` in the same file

Do this.

Add `safety_stock(demand_std, lead_time, z_score=1.65)`. The quantity is $z * \text{demand_std} * \sqrt{\text{lead_time}}$. Demand standard deviation and lead time may be zero. z must be greater than zero. Return the inputs and the formula string with the quantity.

Where this sits.

Safety stock is the other materials tool. It answers 'how much extra do we keep while we wait for the truck?' EOQ answers 'how much do we order each time?' They are different decisions. Both can change a buy, so both will later require approval.

In depth.

`demand_std` is how much demand bounces in one period. `lead_time` is how many of those same periods you wait. z is how rare a stockout you accept. 1.65 is about a 95 percent cycle service level on a normal curve. `safety_stock(10, 4)` with the default z is 33.0, because $1.65 * 10 * \sqrt{4} = 33$. The square root is there because independent periods add variance, and the standard deviation of that sum grows with the square root of the number of periods. Four weeks of wait is not four times the uncertainty. A demand standard deviation of zero means you are sure, so the buffer is zero. A lead time of zero means the material is already here. The docstring states the limit: lead time is treated as fixed. If the mill is sometimes 3 weeks and sometimes 12, this number is too small. The fuller formula needs a lead-time standard deviation. The sample table has `lead_days` and does not have that column, so we do not invent the term.

Tradeoff.

Defaulting z to 1.65 makes the function usable when the question forgets a service level. A silent default smuggles a policy into a buy. The function may use 1.65. The later agent must say that it did. Raising when z is omitted would be stricter and would reject the guide's own safety-stock example.

Check.

`safety_stock(10, 4)['safety_stock']` is 33.0 and `z_score` is 1.65. `safety_stock(10, 4, 0)` returns an error.

Step 8: Write `lookup_sku` and `load_materials_csv`

Do this.

`lookup_sku(inventory, sku)` uppercases and strips the id, returns that row, or returns an error plus the known SKUs. `load_materials_csv(text)` reads the seven required columns and raises `ValueError` on a missing header, a missing column, an empty SKU, a duplicate SKU, or a cell that is not a number.

Where this sits.

The CSV is the project's materials table, not a document to search. The spec agent must not cite it as a specification. The materials agent will call `lookup_sku` and, when exactly one row matches a name like rebar, borrow annual demand and order cost from it.

In depth.

Required columns are sku, description, on_hand, lead_days, unit_cost, annual_demand, and order_cost. csv.DictReader reads a string through io.StringIO, because an upload is already bytes in memory. lstrip of the UTF-8 byte-order mark matters: Excel often writes that invisible character on the first header, and the column would fail the name check even though it looks like sku. Error line numbers start at 2 because line 1 is the header. on_hand and lead_days are ints. Costs and annual demand are floats. A missing SKU is an error dict, not a guessed nearest row. The success path copies the row and sets sku to the normalized key so the result has one spelling. This function does not say 'reorder.' A lookup is a read. Reorder advice is a recommendation and belongs in the agent, where approval can pause it. The loader raises, while EOQ returns an error dict. A bad spreadsheet should fail the upload. A bad number in a chat should finish as a sentence.

Tradeoff.

Skipping a bad CSV row would let the rest of the table load. One typo would then drop a bar and the agent would compute from a quiet hole. Rejecting the whole file is harsher and honest. Pandas would guess types and might turn a bad cell into NaN. The standard-library parser is stricter, which is what you want. Extra columns are ignored, including a column named holding_cost, so a spreadsheet cannot quietly redefine H.

Check.

Loading a header of only sku,description raises ValueError matching 'missing columns'. lookup_sku of ' rebar-5 ' finds REBAR-5. An unknown SKU names the known ids.

Step 9: Prove the math with tests/test_inventory.py

Do this.

Write pytest tests that lock 547.72, reject non-positive EOQ inputs, lock safety stock at 33.0, reject a bad z, look up a SKU case-insensitively, and reject a short or duplicate CSV. Run pytest tests/test_inventory.py -q.

Where this sits.

The test is the spec for this file. Later agents are correct only if they call these functions. If you change the formula, this test fails before a user ever sees a sentence.

In depth.

test_eoq_matches_project2_formula recomputes $\text{round}(\sqrt{300000}, 2)$ rather than only typing 547.72, so the expected value is the formula, not a memorized display. The parametrized test covers demand, order cost, and holding cost at zero or below, and asserts the result has no eoq key. The CSV fixture in the test is two rows, REBAR-5 and AB-34, so lookup and the duplicate-SKU case do not need the full sample pack. Run the file from backend/ with the virtualenv active. Read a failure before you edit the test. The assertion is the contract.

Tradeoff.

Testing through the future HTTP API would prove the wiring and would mix a formula bug with a JSON bug. A direct unit test is the smaller proof. You still need an API test later. You do not start there. Snapshot tests of whole English paragraphs break every time you improve a sentence. These tests lock numbers and error keys.

Check.

pytest tests/test_inventory.py -q is green. Change the formula to $\sqrt{2 * D * H / S}$ and watch it fail. Put the formula back.

Step 10: Cut specifications into chunks in backend/app/rag/chunking.py

Do this.

Write `recursive_split(text, chunk_size=800, chunk_overlap=120)`. Split on blank lines, then newlines, then '.', then spaces. Only a piece that is still too long and has no separator left becomes a sliding character window. Overlap must be smaller than the chunk size.

Where this sits.

A spec is too long to cite as one blob and too structured to slice at a random character. Chunks are what retrieval will rank and what the spec agent will quote. Page numbers are attached later, in ingest. This file only cuts text.

In depth.

Construction specs are already organized by section, paragraph, and sentence. Cutting on those boundaries first keeps 'RFI-014' and a curing rule intact. The Project 2 check is the tiny string `abcdefghij` with chunk size 4, overlap 2, and an empty separator. The windows are `abcd`, `cdef`, `efgh`, `ghij`. A short line such as 'PO-4417 qty 12' stays one chunk under the real 800-character setting. Overlap copies the tail of one chunk onto the head of the next so a rule that crosses the cut still appears in both. People cite page 1. Libraries often count from page 0. The conversion to a 1-based page happens when the chunk is built, not inside the splitter.

Tradeoff.

Semantic chunking asks an embedding model where the topic changes. It can follow meaning, and it can also glue two unrelated sentences that share the word 'shall'. It needs a model you do not want on the critical path of a test. Recursive splitting is inspectable. A chunk size of 800 is a judgment: too small and the citation loses the exception; too large and the citation is a whole section. 800 with 120 of overlap matches the scale of these short sample specs.

Check.

The window test equals those four slices. A two-paragraph string with a small chunk size splits on the blank line and does not glue the paragraphs into one word.

Step 11: Rank chunks with BM25 in backend/app/rag/retriever.py

Do this.

Write `search_chunks(chunks, query, project_id, k=4)`. Keep only chunks whose `project_id` matches. Drop stopwords such as `the`, `where`, and `for` from the query. Score the rest with BM25. Return at most `k` chunks with a positive score, highest first. Break ties by file name and page.

Where this sits.

This is how the spec agent and the risk agent find a passage. The `project_id` filter is the wall between two jobs. Job A must not cite job B even when the words match. The browser never sees this function. FastAPI will call it later.

In depth.

BM25 scores how surprising it is that a chunk contains the query words. A rare token such as RFI-014 is worth more than a common word. The `idf` term is $\ln(1 + (N - df + 0.5) / (df + 0.5))$. The `tf` term saturates so repeating 'steel' ten times does not dominate. `k1` is 1.5 and `b` is 0.75, the usual defaults. Stopwords are removed from the query only. If you leave 'the' in the query, every page that says 'the' earns a positive score and fills the top 4 with junk citations. An empty query or a query that is only stopwords returns no chunks. `k` must be greater than zero. The Chunk records are plain data: `text`, `title`, `page`, `project_id`, `source_name`. No database is required for the test.

Tradeoff.

Embeddings can match 'slab on grade' to 'foundation pour' when the words differ. BM25 cannot, unless both phrases are in the text. Embeddings need a model, can drift when you change that model, and are easy to fake with random vectors. This corpus is a handful of pages full of exact codes: RFI-014, ASTM F3125, SB-21. Lexical search is the right first index. Chroma is the local step when you add a real embedding model. OpenSearch is the shared step after you have an eval that tells you answers got better. Moving to a cluster before that eval exists hides whether the agent or the cluster is wrong.

Check.

A query of 'structural steel' returns the steel chunk and not the gate-hours chunk. The same words in another project_id do not come back. 'where is the' returns nothing. 'where is the steel' returns only the steel chunk.

Step 12: Turn a PDF into chunks in backend/app/rag/ingest.py

Do this.

extract_pages reads a text file as UTF-8 or a PDF with pypdf. If a PDF has no extractable text, raise InvalidDocument. pages_to_chunks runs the splitter and stores a 1-based page number, a title from the first non-empty line, the project id, and the file name on every chunk.

Where this sits.

Upload will call this after the bytes are saved. The original file stays in storage. The chunks are the search copy. They are rebuilt from the files, so the PDF remains the source of truth.

In depth.

Sample specs are short original text, not copied manuals. data/sample holds Division 03 concrete, Division 05 steel, RFI-014, a submittal log, a site logistics plan, and materials.csv. The steel file contains the sentence you will demo: mill lead time is 8 to 12 weeks after approved shop drawings. The RFI contains the line 'Blocked activity: slab on grade foundation pour'. That line is deliberate structure so the risk agent can extract a blocked activity without guessing. A scanned drawing has pixels, not text. pypdf will return an empty string. The product refuses it. Saying 'I cannot read this' is correct. Pretending the drawing was understood is how a made-up spec reaches a superintendent. OCR is a later project. Title text is clipped at 80 characters so a citation label stays a title.

Tradeoff.

Indexing the CSV as if it were a spec would let a keyword search answer 'what is the order cost' from a table row and skip the tool. Then the number is an accident of wording. Keeping the CSV out of the chunk index forces math through calculate_eoq. The cost is that a question phrased only as prose about the table will not cite the CSV. That is the split you want.

Check.

test_sample_pdf_text_survives_a_round_trip finds '8 to 12 weeks' and the blocked-activity line after writing the sample pack and reading it back with pypdf.

Step 13: Store the original file in backend/app/storage/local.py

Do this.

Save uploads at data/runtime/projects/{project_id}/docs/{filename}. Accept only a project id of 1 to 64 letters, numbers, underscores, or hyphens. Keep only the base file name. Allow .pdf, .csv, and .txt. list_files on a missing project returns an empty list.

Where this sits.

This is the laptop version of S3. The key shape `projects/{id}/docs/{filename}` is the same shape the S3 store will use. Agents do not open paths. They ask the store.

In depth.

`Path(name).name` strips directories, so a browser filename of `../../secrets.txt` becomes `secrets.txt` inside the project folder. Spaces become underscores. Characters outside a short safe set are removed. The suffix must be one of the three allowed types, so `file.pdf.exe` is rejected. `check_identifier` rejects `'..'` and slashes as a project id, which is the other half of path safety. `StoredFile` records the name, size, content type, and modified time. modified time is how the engine chooses the newest CSV when the file is not named `materials.csv`. `materials.csv` wins if it exists, so the well-known name is the table even if an older export is still in the folder.

Tradeoff.

A database row per file would give you transactions and queries, and it would duplicate the bytes you already have on disk. For a handful of documents, the folder is the index. The tradeoff is that you must be careful about names. A looser filename rule would accept whatever the browser sends and would also accept a path escape. Strict names are slightly annoying and much safer.

Check.

Uploading `../../secrets.txt` stores `secrets.txt` under the project docs directory and does not create a file next to the data directory.

Step 14: Open the first HTTP door: upload and list

Do this.

Add FastAPI routes `POST /api/projects/{id}/documents` and `GET /api/projects/{id}/documents`. Read the upload in 64 KB pieces and stop if the total passes 10 MB. Validate the bytes before you keep them. A bad PDF or a bad CSV returns a JSON error with no traceback. GET on an unknown project returns an empty document list.

Where this sits.

This is the first time the browser has a door. There is still no chat and no agent. You are proving that a file can enter the platform and be listed. Chat will reuse the same project id.

In depth.

The route lives in `backend/app/api/docs.py`. The size check happens while reading, so a huge body is refused before it is assembled and stored. Empty files are rejected. CSV text is parsed with `load_materials_csv` before save, so a bad table never lands on disk. PDF text is extracted before save, so a scanned sheet never lands either. If save succeeds and the rebuild fails, the new file is deleted. That rollback is why the IAM policy later includes `s3:DeleteObject` on the project prefix only. Errors are `SiteflowError` values with a short code and a detail sentence. The generic exception handler returns 'The request failed.' The server log can keep the traceback. The response cannot. CORS allows `http://localhost:5173` so the Vite app can call the API if the proxy is not used. Health is `GET /api/health` and returns status ok, mode local, use_aws false.

Tradeoff.

Returning a 404 for an unknown project tells the UI that the id was mistyped. Returning an empty list lets a brand-new project look like an empty job, which is what the screen wants on first load. Empty list is the choice here. Streaming the file straight to S3 with a presigned URL is better for a 200 MB drawing and worse for this step, because you would store the bytes before you knew they were a readable PDF.

Check.

pytest tests/test_api.py -q --tb=line -k 'upload or bad_csv or size or health' is green. A .exe upload returns unsupported_file_type. The response text does not contain Traceback.

Step 15: Finish Day 1 by running the file tests together

Do this.

From backend/, run `pytest tests/test_inventory.py tests/test_chunking_and_retrieval.py tests/test_api.py -q`. Do not start the React app. Do not call Ollama. Do not open the S3 console.

Where this sits.

Day 1 is done when a PDF and a CSV can be stored, the CSV can support 547.72, and a second project cannot see the first project's chunks. The graph is the next day because it will trust these results.

In depth.

Read any failure from the assertion upward. If the EOQ test fails, the formula moved. If the BM25 test fails, project filtering or stopwords moved. If the upload test fails, validation or the path rule moved. The sample question you should be able to compute without the server is still $\sqrt{2 * 12000 * 50 / 4} = 547.72$, with the sentence that order cost 50 came from REBAR-5 because the question did not include it.

Tradeoff.

Pushing on to LangGraph while a unit test is red means you will debug a graph that is calling a wrong function. Graphs are harder to read than a 20-line test. Stay on Day 1 until this command is green.

Check.

The pytest command exits 0. You can point at `inventory.py`, `chunking.py`, `retriever.py`, `ingest.py`, `local.py`, and `api/docs.py` and say what each one refuses to do.

Step 16: Define the shared turn in backend/app/agents/state.py

Do this.

Create a TypedDict called AgentState. Fields: message, messages, project_id, route, route_reason, sources, tool_result, needs_approval, recommendation, answer, agents_used, approval. All keys are optional so a node can return only what it changes.

Where this sits.

State is the envelope the supervisor and the specialists pass along one question. It is not the file store and it is not the system of record for the job.

In depth.

Put a fact in state when the next node needs it and the turn is not over: the route, the citations, the tool result, and whether a person still owes a decision. Put a PDF in the document store when every future question on that project needs it. Put the paused turn in SQLite when a human might answer later. Do not put PDF bytes in the checkpoint. Do not keep the approval only in React memory, or a refresh loses it. messages is the short history. message is the current user text the router actually reads. agents_used is the list you will show in the UI so you can say which node ran.

Tradeoff.

A database row for every field is durable and heavy for a single turn. Keeping everything in the HTTP request loses the pause, because the next approval call is a new request. State plus a checkpoint is the middle: small enough to pass between nodes, durable enough to resume.

Check.

You can sort a fact into one of three piles: this turn, the project files, or the paused checkpoint. Sources are dicts of title, page, and snippet, not live file handles.

Step 17: Write the supervisor cascade in backend/app/agents/supervisor.py

Do this.

Write `route_message(message)` so the first matching rule wins. Formula words go to materials, unless the user asked what the spec says. Schedule words go to risk. Document words go to spec. Anything else goes to chat. Return the route and the word that matched. `supervisor_update` writes that onto state and clears the previous answer, sources, and approval.

Where this sits.

This is the switch. Every question hits it first. A wrong switch sends the user to the wrong specialist. The specialists do not second-guess the route.

In depth.

Materials words include `eqq`, `safety stock`, `sku`, `holding cost`, `order cost`, `annual demand`, `on-hand`, and `reorder`. Risk words include `delay`, `risk`, `weather`, `blocked`, `block`, `hold`, and `pour`. Spec words include `lead time language`, `spec`, `rfi`, `submittal`, `division`, `curing`, `mill cert`, `vapor barrier`, and `structural steel`. Patterns use word boundaries so `'hold'` does not match inside `'threshold'`. `'Which open RFIs block the foundation pour?'` contains both `rfi` and `block`. Schedule wins, because the user wants the blocked work, which is the risk agent's job. `'What does the spec say about EOQ?'` contains `eqq` and also asks for the document, so it stays on spec and does not compute a number. Chat is the refusal path: the product will not answer a poem or a sports question from general knowledge. `score_routes` reads `eval/questions.json` and counts how many of the 12 gold questions take the expected route.

Tradeoff.

An LLM classifier can handle a long messy question that matches no keyword. It can also flip between two runs, which makes the eval score wobble and costs a model call on every message. Keywords are stable, free, and explainable: the reason string names the word. Add a model later only for the questions that fall through to chat. A wrong materials route returns a number with an empty source list. A wrong spec route skips the formula and says the documents do not contain it. A wrong chat route refuses a real question. The refusal is the miss you can see. A guessed number is worse.

Check.

`pytest tests/test_routing_and_agents.py -q --tb=line -k 'gold or threshold or document_question' is green.` Accuracy on the 12 questions is 1.

Step 18: Write the spec agent so it can only quote the project

Do this.

In `spec_agent.py`, search this project's chunks. If there are no hits, answer with the refusal that you will not use general construction knowledge. If there are hits, list title, page, and snippet. `needs_approval` stays false. `maybe_paraphrase` may rewrite the draft only when a model is attached, and only when the rewrite adds no new number.

Where this sits.

This is the document specialist. It is what Construction Cloud document search feels like in miniature: an answer tied to a page. Reading a spec does not change the job, so there is no approval card.

In depth.

`compose_spec_answer` builds lines the user can check. Citations are the same snippets, clipped to about 320 characters. `unsupported_numbers` finds digits in a candidate sentence that never appear in the draft. `paraphrase_is_faithful` also requires a few long words from the draft, so a model cannot reply 'ok sure' and pass. If the model throws, the draft is kept. `TemplateChatModel`, the default, returns the draft unchanged. That is why the demo answer for steel contains the words '8 to 12 weeks' from Division 05 and not a fluent paragraph from the model's training data. The search is scoped by `project_id` inside `search_chunks`, and the agent also only receives that project's chunk list.

Tradeoff.

Asking the model to answer from the chunks directly produces nicer prose and sometimes a curing time that was not in the file. Quoting first is clumsier and grounded. A minimum BM25 score would drop weak extra citations. We drop only zero scores, so a second document that shares one real word can still appear. That is visible in the UI and better than a hidden cutoff you cannot explain. Tighten it when the eval says the extra citations hurt.

Check.

An empty hit list does not contain '7 days'. A chunk that belongs to another project does not appear in the answer. A fake model that says 'buy 99999 units' is discarded.

Step 19: Write the materials agent so it will not guess a missing input

Do this.

In `materials_agent.py`, parse D, S, H, demand standard deviation, lead time, z, and a SKU from the sentence. Call `calculate_eoq`, `safety_stock`, or `lookup_sku`. If S or D is missing and exactly one CSV row matches, fill it from that row and say so. If two rows match, ask which SKU. A lookup does not need approval. An EOQ or a safety-stock quantity does.

Where this sits.

This is the tool-using specialist. It is the difference between a chatbot and an agent: a checked function runs, and the paragraph is written from the function's dict.

In depth.

The parser accepts 12,000 with a comma and a dollar sign on the holding cost. Intent is `eoq` when the user says EOQ or gives both demand and holding cost. Intent is `safety stock` when they say safety stock or give a standard deviation. Otherwise a materials question is a lookup. For the guide's rebar question, D is 12000, H is 4, and S is missing. The description token rebar matches only REBAR-5. Order cost 50 is copied from that row. Holding cost stays 4, not the unit cost 4.10. The paragraph includes the formula and the inputs, then says a person should approve the quantity before anyone buys. `needs_approval` is true, so the graph will pause. lookup of REBAR-5 returns on-hand 800 and the sentence 'This is a lookup, not an order.' recommendation is null. Anchor bolts match AB-34 for the annual-demand question and also do not require approval, because no order quantity was computed.

Tradeoff.

A model with function calling can choose the tool from a description. It is flexible, and it will sometimes call EOQ without S. A parser is brittle when someone phrases the question in a new way, and it is testable. Start with the parser. The failure mode is a clarification, which you can see, rather than a silent number. Filling S from the CSV is a convenience. Doing it for two matching SKUs would mix their costs. Refusing the ambiguous case is the stricter and correct rule.

Check.

The rebar EOQ outcome is 547.72, needs approval, mentions REBAR-5, and does not contain 4.10. Two steel SKUs return `ambiguous_sku` and `needs_approval` false. A SKU lookup `needs_approval` is false.

Step 20: Write the risk agent as rules over explicit lines

Do this.

Search the user's words. If that finds nothing, search again with hold, RFI, blocked, weather, and delay added. A line that starts with 'Blocked activity:' means level high and lists that activity. 'on hold' is high. A weather limit with no hold is medium. No text is unknown. High and medium set `needs_approval`. Unknown does not.

Where this sits.

This is the schedule specialist. It tells a superintendent whether a pour is blocked. The level is a rule on text you put in the documents, not a model's opinion about construction.

In depth.

RFI-014 says the pour stays on hold and includes 'Blocked activity: slab on grade foundation pour'. The agent copies that activity into the answer and the recommendation 'Keep this work on hold: ...'. Division 03 says do not place concrete below 40 F or above 95 F. That phrase 'Weather limit' sets medium when no hold is in the hits. The user's words are searched first. Always adding the word RFI would drag the open hold into a question that was only about weather. When both documents really match, the answer mentions the hold and the weather limit, and the level stays high because the hold is the stronger fact. Unknown says it will not invent a schedule risk. There is still no write into a scheduling system. Approval records that a person saw the recommendation.

Tradeoff.

Asking a model 'how risky is this pour?' reads well and cannot be unit tested. A rule on a labeled line is strict: if the document does not contain 'Blocked activity:', the agent will not infer one. You will miss holds that are written as a paragraph. You will not invent holds. For a study product, the miss is the better failure. Add extraction of prose later, beside this rule, and test it.

Check.

A chunk with the blocked-activity line is high and needs approval. A weather-limit chunk alone is medium and needs approval. No hits is unknown and does not need approval.

Step 21: Pause for a person with LangGraph interrupt

Do this.

In `graph.py`, compile a `StateGraph`. Start at the supervisor. Branch to `spec_rag`, `materials`, `risk`, or `respond`. After a specialist, go to `human_review` if `needs_approval` is true, otherwise `respond`. `human_review` calls `interrupt` with the recommendation. On resume it calls `apply_human_decision`. Compile with `SqliteSaver`.

Where this sits.

This is the human-in-the-loop gate. It is the interview sentence: the agent may recommend a buy or a hold, and a person decides before anything is treated as accepted.

In depth.

`interrupt()` raises inside the node and saves the checkpoint. The HTTP layer sees `__interrupt__` and returns `needs_approval` true and status `awaiting_approval`. The next chat on that thread returns 409 until Approve or Reject, so a second question cannot bury an open buy. Resume is `Command(resume={'approved': bool, 'note': str})`. The node runs from the top again, receives that dict, and appends 'Decision: approved.' or 'Decision: rejected.' plus the sentence that SiteFlow does not place orders or change the schedule. If nobody clicks, the checkpoint stays on `human_review`. There is no tool that writes a purchase order, so nothing in a system of record changes. A new process can open the same SQLite file and resume. That is

test_checkpoint_survives_a_new_process. The node functions stay plain. Most tests call them without booting the graph. The graph test proves the pause.

Tradeoff.

A status flag you poll in your own table is easier to see in one SQL query and is a second implementation of what LangGraph already does. interrupt is the library's pause, which is the word you want in the interview. The cost is that you must learn Command(resume) and you must keep a checkpoint. MemorySaver would forget the pause when the process dies. SQLite is the local durable choice. DynamoDB is the same idea when you run more than one server: partition key thread_id, payload blob, optional TTL.

Check.

An EOQ chat returns awaiting_approval and agents that include human_review. A second chat returns 409 approval_pending. Reject returns a resolved answer containing 'Decision: rejected.' A new Engine on the same SQLite file can still approve.

Step 22: Expose chat and approvals over HTTP

Do this.

POST /api/chat accepts project_id, thread_id, and message. Empty or whitespace messages are 422. POST /api/approvals/{thread_id} accepts approved and an optional note of at most 500 characters. Unknown threads are 404. A thread that is not waiting is 409. A thread cannot move to a different project id.

Where this sits.

These two routes are the only way the screen talks to the graph. The screen does not import LangGraph. Pydantic is the contract: ChatRequest, ChatResponse, ApprovalRequest.

In depth.

ChatRequest strips the message and rejects a blank one before any node runs. Identifiers use the same 1-to-64 character rule as file storage. The engine loads the checkpoint. If the graph is still on human_review, it refuses a new question. If the stored project_id differs, it returns thread_project_mismatch. That is isolation for conversations, matching the chunk filter for files. The response always includes answer, sources, agents_used, needs_approval, and recommendation. It also includes route, route_reason, status, and tool_result so you can see the math. Extra fields are fine. The screen must keep working if it only reads the contract fields. Logs include request_id, project_id, route, and latency_ms. They do not include a traceback in the JSON. Replacing a text file and asking again must drop the old sentence. That proves the index was rebuilt from disk rather than stuck in memory.

Tradeoff.

Server-sent events would show tokens as they arrive. The guide's API section that the screen must use is a single JSON body. Streaming is a later format on the same graph. Shipping both now would double the client. JSON first. A missing thread could return an empty chat. 404 is clearer: you cannot approve a conversation that never started.

Check.

The steel question returns route spec, needs_approval false, and the text '8 to 12 weeks' with a source page. The poem returns route chat, no sources, and the refusal. A blank message is 422 and the body does not contain Traceback.

Step 23: Score the 12 gold questions in eval/questions.json

Do this.

Keep 4 spec questions, 4 materials questions, 2 risk questions, and 2 out-of-scope questions. Each row has an id, a bucket, an expected_route, and the question text. score_routes must report accuracy 1.

Where this sits.

This is the evaluation harness. Routing accuracy answers 'did the right specialist run?' The API tests answer a second question: 'is the demo sentence supported by the file or the formula?'

In depth.

The spec questions include the steel lead-time sentence, Division 03 curing, mill certs, and the vapor barrier in the submittal log. The materials questions include the rebar EOQ, a SKU lookup, safety stock with std 10, lead time 4, and z 1.65, and the anchor-bolt lookup. The risk questions are the blocked pour and the weather delay. The out-of-scope questions are a poem and a sports question. You measure routing with score_routes. You measure faithfulness on the demo answers: the steel answer contains '8 to 12 weeks' and a source; the EOQ is 547.72 and needs approval; the pour names 'slab on grade foundation pour'; the poem has an empty source list. Latency is logged as latency_ms and is not yet graded. A correct citation means the cited page contains the claim. Precision is the share of cited claims that pass. Recall is the share of true answering passages you actually cited. Embedding drift is not measured, because this slice has no embedding model. The method, when you add one, is a frozen set of query and chunk ranks compared before and after the model change.

Tradeoff.

Hand-scoring every answer is slow and catches prose the tests miss. Automatic routing accuracy is fast and blind to a right route with a wrong sentence. You want both, and you already have routing fully scored plus the demo sentences tested end to end. Scoring all 12 answers for faithfulness is the next eval, not a reason to skip the 12 routes.

Check.

test_gold_questions_all_route_correctly sees the bucket counts 4, 4, 2, 2 and accuracy 1.

Step 24: Point the sentence writer at your local Ollama

Do this.

Keep routing on the keyword cascade. When you want prose, construct OllamaChatModel('llama3.1:8b') and pass it as the model into compile_graph. Leave it unset for tests. The number guard stays in front of whatever model you pass.

Where this sits.

This is the optional last inch of the agent, not the agent itself. The graph already produced a grounded draft. Ollama may rephrase it. Bedrock implements the same invoke method for the day you leave the laptop.

In depth.

OllamaChatModel posts to http://127.0.0.1:11434/api/chat with stream false and reads message.content. The test uses httpx.MockTransport, so pytest does not need the daemon. BedrockChatModel sends the Anthropic Messages body: anthropic_version bedrock-2023-05-31, system as a top-level field, and user text in content blocks. You can explain that body without an AWS call because the builder is a pure function. Use keywords for routing. Use the 8B model, if you use a model at all, to polish a draft. Use a larger model only for that polish, still behind unsupported_numbers. Spending a large model on every route burns money to do a job the cascade already does. If Ollama is down, maybe_paraphrase catches the exception and returns the draft, so the demo still answers.

Tradeoff.

Template answers are repetitive and auditable. Ollama answers are smoother and can drift. The guard deletes a paraphrase that introduces 99999. It will not catch a false sentence that uses only numbers already in the draft, such as swapping which document a number came from. That remaining risk is why the UI still shows the source snippet beside the prose.

Check.

With no model, the EOQ sentence is the template. A fake Ollama that returns a sentence containing 547.72 and words from the draft is accepted. A fake that returns 99999 is rejected. ollama list still shows llama3.1:8b when you want to try the live daemon yourself.

Step 25: Build the React screen that only calls FastAPI

Do this.

In frontend/, the Vite app proxies /api to port 8000. client.js exports uploadDocument, listDocuments, sendChat, and submitApproval. The page lists files, shows the chat, shows source chips, and shows an approval card when needs_approval is true. While the card is open, Send stays labeled Send and a line says to resolve the card. It does not say Working.

Where this sits.

This is the product side of the picture. It is the demo a person can click. It is also the proof of the platform split: search the frontend folder and you will not find a call to S3, LangGraph, or Ollama.

In depth.

Run the API with uvicorn app.main:app --port 8000 from backend/, and npm install && npm run dev from frontend/. Open http://localhost:5173. The first load lists project demo, which the server seeds from data/sample. Ask the steel question and read the citation. Ask the rebar EOQ question and read 547.72 on the card. Reject it. The answer grows a decision line and says the app does not place the order. That sentence is product work. It stops you from demoing a write you did not build. Errors show the JSON detail field. Changing the project id starts a new thread, because a thread is bound to one project. Project 2 was Streamlit. This loop wants a real frontend on an API. Python remains the place the behavior lives, which is why the tests are pytest.

Tradeoff.

Streamlit would ship faster and would hide the contract inside the Python process. React is more files and makes the four API calls obvious. Tailwind is optional and was skipped so the screen depends only on React and Vite. A component test in the browser would be a second harness. The API tests already lock the JSON. Click the three questions once so you know the screen matches.

Check.

The steel answer shows 8 to 12 weeks and a source. The EOQ card shows 547.72. Reject shows 'Decision: rejected.' The page never shows the word Traceback.

Step 26: Add the S3 and IAM seam without pretending you deployed it

Do this.

Study backend/app/aws/s3.py and infra/iam-siteflow.json. The S3 store has the same save, read, list, and delete methods as the disk store. put_object sets ServerSideEncryption to AES256. presigned_put_url expires in at most 300 seconds and refuses keys outside projects/{id}/docs/. build_engine raises if USE_AWS is true, so the app does not silently open a real client.

Where this sits.

This is the same file job as Step 13, with AWS authentication. You explain it from the tests. You turn on a real bucket only when you choose to, with keys in .env, in us-west-2.

In depth.

Block public access on the bucket when you create it. Enable default encryption. The IAM actions are s3:ListBucket on the one bucket, limited by a prefix condition to projects/*, plus s3:GetObject, s3:PutObject, and s3:DeleteObject on arn:aws:s3:::siteflow-docs-YOURNAME/projects/*. Delete is not in the guide's minimum list. It exists so a failed rebuild can remove the object it just wrote. There is no s3:*, no iam:*, and no Bedrock permission until you actually call Bedrock. There is no explicit Deny. Omission protects you only when this user has no second policy that allows more. A sloppy Deny on s3:* would also cancel the Allows. The running UI still uploads multipart through FastAPI, because that is the contract and because the server can reject a bad PDF before storage. A presigned PUT is the better path when the file is large: the API should not hold a 200 MB drawing in memory. Five minutes is long enough for a normal PDF and short enough that a forwarded link dies. Keep the bucket, Bedrock, and the app in us-west-2. A model call in us-east-1 against a bucket in us-west-2 is two regions and sometimes a model that is not enabled where you called. On ECS later, FastAPI sits behind an ALB, the built React app sits on S3 and CloudFront, documents sit on S3, and checkpoints move to DynamoDB. The task is stateless only after SQLite is gone. Do not start with EKS. A 200 MB package should land in S3 and be parsed by a worker on ObjectCreated, not inside the request.

Tradeoff.

Wiring a live bucket during study proves IAM and creates a bill, a region, and a key-management problem while you are still learning the agent. Fake-client tests prove the calls: SSE-S3, pagination, the presign cap, and the IAM action set. Say clearly in an interview that the laptop path is what you run, and the S3 class is the swap you have tested without a network. Claiming the agent already writes RFIs into Autodesk Construction Cloud would be false.

Check.

test_iam_policy_is_limited_to_one_bucket_prefix sees only List, Get, Put, and Delete.
test_use_aws_flag_does_not_silently_open_a_real_client raises.
test_presign_is_capped_at_five_minutes_and_stays_under_the_prefix rejects 301 seconds and a key outside the prefix.

Step 27: Run the three demo questions and say the walk-through out loud

Do this.

Start the API and the Vite app. Ask the steel lead-time question, the rebar EOQ question, and the blocked-pour question. Reject or approve the cards. Then close the laptop lid on the code and explain one question hop by hop.

Where this sits.

This is the definition of done for the slice. The PDF's earlier steps exist so this demo is something you understand, not something you only clicked.

In depth.

Steel path: Chat.jsx calls sendChat. The browser POSTs /api/chat. Vite forwards it to FastAPI. Pydantic accepts the message. The supervisor matches 'lead time language' and chooses spec. BM25 on project demo ranks Division 05 first. The answer quotes '8 to 12 weeks'. needs_approval is false. SQLite stores the turn. Sources.jsx shows the title and page. S3 is not on this hop while USE_AWS is false. Say that. EOQ path: the cascade matches eoq and holding cost. The parser reads D=12000 and H=4. S comes from REBAR-5 and the answer says so. calculate_eoq returns 547.72. needs_approval sends the graph into interrupt. The card appears. POST

/api/approvals resumes with the boolean. The materials table does not change. No order is sent. Pour path: risk matches block and pour, finds the blocked-activity line, and pauses for the same reason: the recommendation can stop work. If you never click, the thread stays paused and the next question is a 409. Out of scope, a poem has no sources. What this slice still does not do: it does not call your AWS account unless you later opt in, it does not OCR a scanned sheet, it does not create an RFI, and it does not run OpenSearch, Lambda, or EKS.

Tradeoff.

Memorizing a script without the tests means the first follow-up question breaks you. The tests are how you know which sentence is safe to say. Skip any interview sentence you cannot point at in a file. Use your own field story for the behavioral question: a time you decided with incomplete information, and where you would put the human gate if someone wanted the automation to place the order itself.

Check.

pytest -q from backend/ is green. You can walk the steel question and the EOQ question without reading this PDF. You can say what is not built.

After the last step

From `projects/siteflow/backend`, with the `virtualenv` active, `pytest -q` should be green. From `projects/siteflow/frontend`, `npm run dev` serves the screen on port 5173. Ask these three questions on project demo:

```
What is the lead time language for structural steel?  
If rebar demand is 12,000 units and holding cost is $4,  
what EOQ should we use?  
Which open RFIs block the foundation pour?
```

The first answer cites 8 to 12 weeks. The second is 547.72 and waits for approval. The third names the slab-on-grade pour and also waits. Rejecting a card records the decision. It does not place an order and it does not edit a schedule.